



Norwegian University of
Science and Technology

Formally Verified Reduction Proof for the Free XOR Garbling Scheme

Prateek Sharma

Submission date: July 2026
Main supervisor: Tjerand Silde, NTNU
Co-supervisors: Russell W. F. Lai and
Chris Brzuska, Aalto University

Norwegian University of Science and Technology
Department of Information Security and Communication Technology

Problem description

Garbled circuits are a fundamental cryptographic primitive used in secure two-party computation (2PC), enabling two parties to jointly compute a function without revealing their private inputs—only the output is revealed. The concept was introduced by Andrew Yao in 1986 [Yao86], where functions are expressed as Boolean circuits and each wire is labelled with encrypted wire keys. Early constructions required four ciphertexts per gate, resulting in significant communication overhead and limiting practical adoption.

Subsequent optimizations such as Point-and-Permute, Half-Gates and the Free XOR technique greatly reduced the number of ciphertexts needed per gate, as well as reducing the number of gates to be transmitted, leading to substantial performance improvements [BMR90; ZRE15; KS08]. These optimizations form the basis of many state-of-the-art secure computation frameworks.

Formal verification of cryptographic constructions is essential to ensure correctness and prevent subtle vulnerabilities arising from unchecked corner cases. Brzuska and Oechsner [BO23] developed a state-separating pen-and-paper reduction proof for Yao’s original garbling scheme. State-separating proofs model protocol components as isolated “packages” with private state and oracle interfaces, allowing for structured security reductions. Their work established security by reducing the garbling scheme to the IND-CPA security of the encryption primitive used for gate tables.

Domino is an automated verification tool specifically designed for state-separating proofs. It has been used to reason about complex real-world protocols such as TLS [Raj25] and WPA2 [BDE+22]. Domino offers a high level of automation and readability because proof scripts resemble the pseudocode found in traditional cryptographic proofs.

Furthermore, the above mentioned proof for Yao’s garbling scheme by Brzuska and Oechsner [BO23] was later formalized in Domino [BW]. It can serve as a good starting point for the thesis to observe how games may be converted from pen and paper proofs to Domino code.

The goal of this thesis is to formalize and mechanize a security reduction for the Half-Gates garbling scheme within the Domino framework. This includes:

- Developing a state-separating, pen-and-paper reduction proof for the Free XOR technique.
- Encoding this proof in Domino.

This work extends formal verification techniques to an optimized and widely deployed garbling construction and demonstrates Domino’s applicability to modern

cryptographic primitives.

Approved: 2026-03-09 – Tjerand Silde, NTNU (Main supervisor)

Abstract

Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

This is the second paragraph. Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

And after the second paragraph follows the third paragraph. Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

After this fourth paragraph, we start a new paragraph sequence. Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of

the original language. There is no need for special content, but the length of words should match the language.

Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

Preface

Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

Contents

List of Figures	vii
List of Tables	ix
List of Algorithms	xi
1 Introduction	3
1.1 Motivation	3
1.2 Objectives	4
1.3 Sustainability	4
2 Background	5
2.1 Garbled Circuits	5
2.2 Garbling Schemes	7
2.2.1 Properties	7
2.2.2 Defining Privacy	8
2.3 Optimizations to Yao's Garbling Scheme	9
2.3.1 Point and Permute	11
2.3.2 Garbled Row Reduction - 3	12
2.3.3 Free XOR	12
2.3.4 Half-Gates	12
2.4 Formal Verification and Tools	12
2.4.1 Domino	12
3 Pen-and-Paper State-Separating Proof	13
3.1 State Separating Proofs	13
3.2 Existing Half-Gates Proof and how it is to be modelled into state separating proofs	14
4 Domino	15
4.1 What is Domino?	15
4.2 Using Domino	15
4.2.1 Dependencies	15

4.2.2	Project Structure	16
4.3	Coding Half-gates in Domino	16
4.4	Improvements	16
5	Discussion	17
6	Conclusion	19
	References	21

List of Figures

2.1	Garbled Circuit Protocol	6
2.2	PrvInd and PrvSim Games - adapted from [BHR12]	8
2.3	An AND gate and its garbled form.	10
2.4	Garbled table for an AND gate	11
2.5	Garbled table for an AND gate with Point and Permute	12

List of Tables

List of Algorithms

TODO: Remember to change minus to hyphen where needed.

Chapter 1

Introduction

1.1 Motivation

Secure Multiparty Computation, or just Multi-Party Computation (MPC) refers to a class of problems in which two or more parties want to compute a common function using their individual inputs, but do not want to share these inputs with each other. Common use cases of MPC today include running heavy computation on cloud servers, or querying third party datasets that cannot be shared directly.

Since remote servers are often owned and operated by a separate entity, it is difficult to obtain the same security and privacy guarantees that a user's own machine can provide. Besides user requirements, cybersecurity regulations also require methods to secure the data being processed from the data processor, such as in the case of private healthcare data, or other forms of personal data [PtC16].

Two-party Computation (2PC) is a special case of MPC in which only two parties are involved in the protocol. Often attributed to Andrew Yao's seminal work in 1986 [Yao86], the first approach for 2PC was to represent the function to be computed as a boolean circuit, and to use a Garbled Circuit (GC) to compute the output of the function, while maintaining the secrecy of the inputs of both parties. For more than a decade after this work, Garbled Circuits were considered to not be practical because of the high bandwidth and computation costs of garbling, and due to the fact that garbling an arbitrary function into an optimized boolean circuit is difficult. However, there has been ongoing work in the field that has led to optimizations that allow them to be somewhat usable in generic 2PC systems like Fairplay [MNPS04], and in niche applications such as smart contracts [HYLL].

In this work, we aim to convert the proof of one such optimization (Free XOR) [KS08] to a State-Separating Proof (SSP) format [BDF+18], and formally verify the correctness of this converted proof. Formal verification allows us to base the correctness of a protocol or scheme on mathematical and logical constructs, rather

than exhaustively testing edge cases, as is often the case in software engineering. We use Domino, a proof verification tool for SSPs, which is currently in development, and examine the suitability of its current feature set for verifying complex proofs, while also attempting to validate the correctness of our Free XOR State-Separating Proof.

The initial goal of this project was to verify the Half-Gates garbling scheme [ZRE15]. However, since the Half-Gates scheme works under the Free XOR assumption, it was mandatory for us to first solve the same problem for the Free XOR scheme. Due to the limited timeframe of the project, we were unable to complete both, and leave Half-Gates as future work. We describe the details of this in further sections.

The rest of this paper is organized as follows: Chapter 2 contains background information on garbling schemes, optimizations, and Domino. Chapter 3 details the converted pen-and-paper proof for Free XOR. Chapter 4 contains further details about the proof tool Domino, a brief overview on how to set it up, and an encoding of the State-Separating Proof in Domino. Chapter 5 discusses the advantages and challenges of working with Domino as opposed to plain pen-and-paper SSP. Chapter 6 contains concluding remarks and future work.

1.2 Objectives

The initial goals of this thesis were as follows:

1. Developing a State-Separating, pen-and-paper reduction proof for the Half-Gates garbling scheme.
2. Encode and verify this proof in Domino.

While working on the proof for the Half-Gates scheme, it was revealed that it is not possible to avoid proving (by assumption, or other means) the Free XOR scheme that Half-Gates depends on without functional and efficiency losses in Half Gates. A State-Separating Proof for Free XOR has not been worked on in the literature we reviewed, and we decided that it was fair to prove the Free XOR scheme instead, as it would allow future work in Half-Gates.

1.3 Sustainability

TODO

This project follows the United Nations Sustainable Development Goals №x and y.

Chapter 2

Background

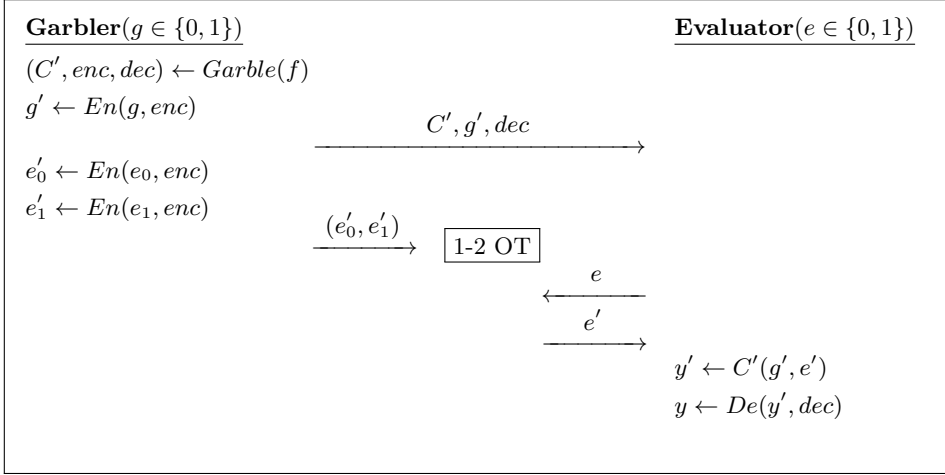
In this chapter, we discuss the basic concepts that the reader would need to know in order to understand the Free XOR garbling scheme, as well as State-Separating Proofs.

2.1 Garbled Circuits

A Garbled Circuit can be informally defined as a two-party protocol, in which both parties want to evaluate a shared function C together, without revealing their inputs to each other. The two parties participating in the protocol are called the Garbler (or the Garbled Circuit generator), and the Evaluator. The function C needs to be expressed as a boolean circuit if a Garbled Circuit protocol is used. Input hiding is achieved using something called *garbling*, which in practice is some form of encryption. Similarly, the final output is initially garbled, and needs to be ungarbled, which is then implemented using a corresponding decryption function.

A simple run of the protocol starts with both parties possessing their own input bit respectively (g , and e). The protocol proceeds as follows (ref. Figure 2.1):

1. The Garbler garbles the circuit C to obtain the Garbled Circuit C' , along with some encoding information (enc) and decoding information (dec). These will be used to garble and ungarble inputs and outputs.
2. The Garbler garbles its own input g into g' .
3. The Garbler then sends across C' , g and dec to the Evaluator.
4. The Evaluator still needs to garble its own input, but does not possess the encoding information, enc to do so. Therefore, both parties participate in what is called a 1-out-of-2 Oblivious Transfer (OT) protocol [EGL82]. This protocol (adapted for GCs) requires the Garbler to garbled both possible bit-values that the Evaluator may possess, and provide them to the OT protocol. Then, the

**Figure 2.1:** Garbled Circuit Protocol

Evaluator provides their chosen bit e to the OT protocol and the OT protocol returns the garbling of that bit (e'). The Garbler remains unaware of which garbling was sent to the Evaluator, and the Evaluator remains unaware of the value of the other garbling.

5. At this point, since the Evaluator possesses both garbled inputs (g' and e') and the Garbled Circuit (C'), it can compute the garbled output, y' by running the garbled inputs through the Garbled Circuit.
6. Finally, using the decoding information dec , the Evaluator can obtain y by ungarbling y' . This output is equivalent to the output of the function $C(g, e)$.
7. If necessary, the Evaluator can send the final output back to the Garbler.

The protocol can also be scaled up to support larger inputs from both parties. However, this would increase the complexity of the Oblivious Transfer protocol.

Both the Garbler and Evaluator are assumed to be *honest-but-curious*. This means that both parties adhere strictly to the protocol specification. However, they try to learn more than what is intended by using the information received from running the protocol, and from their own randomness and state. So, as an example, the Evaluator could try to obtain the Garbler's bit by ungarbling it, but will not succeed since the algorithm dec that the Evaluator receives does not ungarble the inputs.

This kind of adversary is equivalent to the 'Semi-Honest' adversary described by Aumann and Lindell in [AL10].

2.2 Garbling Schemes

In the previous section, we discussed a 2PC Garbled Circuit protocol. However, it is easier to reason about a part of the security of such a protocol using a more generalized construct called a garbling scheme. A garbling scheme leaves out the Oblivious Transfer sub-protocol and focuses on the correctness and security properties of the garbling of the circuit and the input.

A garbling scheme [BHR12] can be expressed as a 5-tuple of algorithms

$$(\text{Garble}(\text{Gb}), \text{Encode}(\text{En}), \text{Decode}(\text{De}), \text{GarbledEval}(\text{Ev}), \text{Eval}(\text{ev}))$$

1. $\text{Garble}(\text{Gb})$ takes as input the function f and the security parameter 1^λ and returns (F, e, d) , where F is the Garbled Circuit, e is the encoding information and d is the decoding information.
2. $\text{Encode}(\text{En})$ takes as input the initial input x and the encoding information e , and returns the garbled input X .
3. $\text{GarbledEval}(\text{Ev})$ takes as input the Garbled Circuit F and the garbled input X , and returns a garbled output Y .
4. $\text{Decode}(\text{De})$ takes input the garbled output Y and the decoding information d , and returns the ungarbled output y .
5. $\text{Eval}(\text{ev})$ takes as input the function f and the plain input x and returns $f(x)$. We will just substitute this with $f(x)$.

Relating this to the representation of the Garbled Circuit protocol in the previous section, the inputs g and e from the previous section are both shares of x . That is, x can be expressed as $x = g \parallel e$.

2.2.1 Properties

The correctness property of a garbling scheme describes its intended behavior. It says that if $(F, e, d) \leftarrow \$ \text{Gb}(1^\lambda, f)$,

$$\forall x, \text{De}(\text{Ev}(F, \text{En}(x, e)), d) = f(x)$$

Besides correctness, a garbling scheme can have one or more of the following security properties [BHR12]:

1. **Privacy:** Knowledge of (F, X, d) does not reveal anything beyond $f(x)$. This means that it is impossible for an Evaluator to obtain the initial input of the Garbler .

2. **Obliviousness:** Knowledge of (F, X) does not reveal anything. This means that despite being able to solve the Garbled Circuit, the solver would not be able to map the garbled output to the real output without d .
3. **Authenticity:** Knowledge of (F, X) does not allow one to find a $Y' \neq \text{Ev}(F, X)$ such that a d exists to solve $(De(d, Y') \neq \perp)$.

Of these properties, **Privacy** is the one that is most important to us in this work, and it is what we will continue working with in the rest of the paper.

2.2.2 Defining Privacy

In [BHR12], Bellare, Hoang and Rogaway define privacy as two different cryptographic games: an *indistinguishability* based one (Game $\text{PrvInd}_{G, \Phi}$), and a *simulation* based one (Game $\text{PrvSim}_{G, \Phi, S}$) as described in Figure 2.2.2. In these notations, and elsewhere, Φ refers to the side information function, which describes how much information about the circuit is public. In common usage, it is often either the whole circuit (denoted as $\Phi_{\text{circ}}(C)$), the topology of the circuit ($\Phi_{\text{topo}}(C)$), or the size of the circuit ($\Phi_{\text{size}}(C)$).

$G_{\text{PrvInd}_{G, \Phi}}^b$	$G_{\text{PrvSim}_{G, \Phi, S}}^b$
<u><u>GARBLE(f_0, f_1, x_0, x_1)</u></u>	<u><u>GARBLE(f, x)</u></u>
if $\Phi(f_0) \neq \Phi(f_1)$ then	if $b = 0$ then
return $\perp$	$(F, e, d) \leftarrow Gb(1^\lambda, f)$
if $f_0(x_0) \neq f_1(x_1)$ then	$X \leftarrow En(x, e)$
return $\perp$	else
$(F, e, d) \leftarrow Gb(1^\lambda, f_b)$	$y \leftarrow ev(f, x)$
$X \leftarrow En(x_b, e)$	$(F, X, d) \leftarrow S(1^\lambda, y, \Phi(f))$
return (F, X, d)	return (F, X, d)

Figure 2.2: PrvInd and PrvSim Games - adapted from [BHR12]

In the indistinguishability based game (Game $\text{PrvInd}_{G, \Phi}^b$), the GARBLE oracle takes as parameters two pairs of functions and corresponding inputs, (f_0, x_0) and (f_1, x_1) such that $f_0(x_0) = f_1(x_1)$, and the bit b that defines which (f, x) pair GARBLE will use. A distinguishing adversary $\mathcal{D}$ tries to guess the bit-value b that was used to generate the (F, e, d) tuple. A INITIALIZE procedure samples the value of b and it is unknown to the adversary. GARBLE then uses the selected (f, x) to generate F , X and d and returns it to the adversary. The advantage of the distinguishing adversary here is defined as the difference in the probability that the adversary guesses the

correct bit and the probability of guessing randomly ($P = 1/2$).

If b' is $\mathcal{D}$'s guess,

$$\text{Adv}_{D, G_{\text{PrvInd}}^0, G_{\text{PrvInd}}^1}(\lambda) = |\Pr[b' = b] - \frac{1}{2}|$$

rewrite next 2 paras for clarity

However, most follow-up works such as Half-Gates [ZRE15] and Free XOR [KS08; App16], use the simulation based paradigm (the PrvSim game from Figure 2.2.2). Since privacy here means that the running the garbling and encoding algorithms Gb and En does not reveal anything about f or x , we let the simulator $\mathcal{S}$ access everything that someone participating in the protocol (semi-honest party) would be able to see, i.e., the security parameter (1^λ), the output of the algorithm y , and the information function of the circuit $\Phi(f)$. Then, this simulator tries to generate a $(\bar{F}, \bar{X}, \bar{d})$ tuple whose distribution is computationally indistinguishable from the one generated by actually running Gb and En .

If the distribution of $(\bar{F}, \bar{X}, \bar{d})$ is actually indistinguishable to an adversary $\mathcal{A}$ from (F, X, d) , an adversary cannot claim to have recovered anything more than what the simulator knew, and hence the privacy of f (except $\Phi(f)$) and x is preserved. For further reading on simulators, Lindell provides a comprehensive overview of simulators for beginners in [Lin17].

2.3 Optimizations to Yao's Garbling Scheme

Before we get into the optimizations for garbling scheme, we must first define a base. In this work, we consider the garbling scheme model defined in [LP09], and will refer to it as Yao's garbling scheme. The garbling of a circuit f consists of n garbled gates, where n is the number of gates that compose the original circuit before garbling. Each (garbled and ungarbled) gate has two input wires (A and B) and one output wire (C). Each of these ungarbled wires can have two possible values on each wire - 0 or 1 (corresponding to real-life boolean circuits). We will use a , b , and c to refer to the actual bit-values of each wire. Figure 2.3 shows this as a diagram, and the truth table of an AND gate.

Now, we map each of the possible values on each wire to a random string of length λ denoted by the wire name and the bit-value it was mapped to. This random value is called the wire label for that particular wire and bit-value combination. For example, the random value mapped to the 0-input on wire A would be called A_0 . After repeating this process to get all six wire labels ($A_0, A_1, B_0, B_1, C_0, C_1$), we can generate the garbled table. It is worth noting that this mapping from the wire's bit-value to the wire label corresponds to the encoding information e used in the En function in the description of a garbling scheme in Section 2.2.

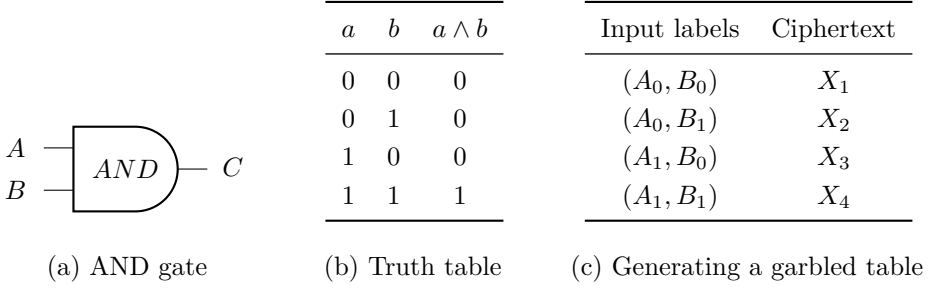


Figure 2.3: An AND gate and its garbled form.

The garbled table contains 4 rows, like the truth table, and it contains encryptions of the wire label of the output value of each row ($C_{a \wedge b}$), under two keys - the wire labels of the corresponding inputs (A_a, B_b) that generated the output $a \wedge b$ in that row of the truth table. For example, if we consider row 1 of Figure 2.3(c), we would have

$$X_1 = \text{Enc}(A_0, \text{Enc}(B_0, C_0)).$$

We know from the protocol description in Section 2.1, that besides the garbling of the circuit, the Garbler also sends the "garbling" of the inputs to the Evaluator. These garblings of the input refers to the wire labels corresponding only to the inputs that the parties want to evaluate using the circuit. Thus, the Evaluator receives the garbled table, as well as the wire labels of the inputs (which, for a single wire, can either be 0 or 1, but not both simultaneously). We observe from Figure 2.4 that decrypting any row requires possession of a specific A wire label and a specific B wire label. This means that if the Evaluator only receives only one A label and only one B label, they cannot decrypt any row other than the one that was encrypted using those labels since at least one of the two decryptions will fail. This preserves the privacy of the original bit-value of wire C . However, this requires that the Garbler provides a mapping from the final output wire labels to the real bit-value of that wire. This would correspond to the decoding information d used in the function De .

In this garbling scheme, we assume that when trying to decrypt a ciphertext without the right keys, the decryption will fail and the decryptor will know that it has failed. This can be achieved by concatenating some specific, publicly-known padding in the wire labels for wire C . This is related to a property called special correctness that where attempting to decrypt a ciphertext without a valid key will result in a decryption failure message with overwhelming probability. This character $\perp$ is commonly used to show such a result in cryptographic notation. Special correctness is a property used later in our work as well.

Ciphertext	Value
X_1	$\text{Enc}(A_0, \text{Enc}(B_0, C_0))$
X_2	$\text{Enc}(A_0, \text{Enc}(B_1, C_0))$
X_3	$\text{Enc}(A_1, \text{Enc}(B_0, C_0))$
X_4	$\text{Enc}(A_1, \text{Enc}(B_1, C_1))$

Figure 2.4: Garbled table for an AND gate

For a larger circuit, garbled tables are generated for each garbled gate in the original circuit, and all of them are transmitted to the Evaluator. In addition to this, a larger circuit would have a larger input as well, so the wire labels corresponding to every input bit is also sent.

2.3.1 Point and Permute

On observing Yao's garbling scheme, it quickly becomes obvious that the Evaluator may have to decrypt 4 rows of the garbled table in order to receive the correct output wire label. Yao's garbling scheme also requires special correctness, which is not difficult to implement, but there is a way to avoid the whole problem in the first place. This technique, now called Point and Permute, is based on the value λ_{W-1} defined in the protocol in [BMR90].

To put simply, for each wire W in the circuit, with wire labels W_0 and W_1 , each of length λ , an extra bit is sampled (called the permute or color bit). Let us call this bit l_W . Then, for each wire W ,

$$W_0 = W_0 \| (l_W), \text{ and}$$

$$W_1 = W_1 \| (l_W \oplus 1)$$

Now, when generating a garbled table, the ciphertexts are ordered based on the permute bit of the wire labels used to encrypt it. For example, if we say that the encryption generated by wire labels A_a and B_b would be in row $2l_a + l_b + 1$, and if A_0 and B_0 both have the permute bit 1, the garbled table would be ordered as in Figure 2.5. Then, the Evaluator can see the last bits (the permute bits) of both input wire labels and identify which row to decrypt in order to obtain the output wire label. This process continues until the end of the circuit is processed since each output wire label contains a permute bit as well, and all the garbled tables are ordered based these permute bits. However, this requires that the Evaluator and

l_a	corresp. A_a	l_b	corresp. B_b	C_c based on (A_a, B_b)	Ciphertext
0	A_1	0	B_1	C_1	$\text{Enc}(A_1, \text{Enc}(B_1, C_1))$
0	A_1	1	B_0	C_0	$\text{Enc}(A_0, \text{Enc}(B_1, C_0))$
1	A_0	0	B_1	C_0	$\text{Enc}(A_1, \text{Enc}(B_0, C_0))$
1	A_0	1	B_0	C_0	$\text{Enc}(A_0, \text{Enc}(B_0, C_0))$

Figure 2.5: Garbled table for an AND gate with Point and Permute

Garbler use the same method of permuting the rows, which can be defined in the protocol specification.

This method reduces the number of ciphertexts to be decrypted by the Evaluator to 1, which makes the evaluation faster.

2.3.2 Garbled Row Reduction - 3

Compatible with Free XOR

2.3.3 Free XOR

2.3.4 Half-Gates

Mention that this was the original objective of the thesis, however work with this requires freexor to be completed

2.4 Formal Verification and Tools

Start from Formal verification in general (read more too), and then go into tools used for formal verification in cryptography.

2.4.1 Domino

How is it different from easycrypt? why did we choose this? what is a state separating proof and why is this more suited to those?

Chapter 3

Pen-and-Paper State-Separating Proof

Read half gates and chris's paper to understand what to do here even. (WIP) This chapter introduces the notion of a State Separating Proof, and how it differs from a traditional Game-playing proof. **is game-playing the correct term here?** Then, we define our pen-and-paper State-Separating Proof for the Free XOR garbling scheme here.

3.1 State Separating Proofs

- What are games and packages
- Something about why state separating proofs exist. "composable" and "constructive" security
- State separating proofs allow us to make intermediate arguments (hybrids) in a reusable manner. It is often more intuitive to model something as its own package that has inputs and outputs than to have it be part of many different codes and realise that its the same thing.

Composability of layers is very important so we can derive circuit security from just proving layer security. The simulator for layer n must output a garbling of its output which is then supplied to a new simulator for layer $n+1$. Therefore, if a simulator can simulate a gate by getting its inputs (garbled input) and return garbled output, it can be composed with the simulator for the next gate and so on. Same idea for layers.

Actually, to add to the above, the simulator does NOT know the input. It only gets the circuit information loss of the garbled circuit $\Phi(C)$, the output of the garbled circuit Y , and the security parameter 1^λ . So it samples a random input label (in this case its the label, in the original Yao paper it was a random bitstring sampled for the wire key, but since we are working in a setting where the Free XOR optimization is assumed, we will probably end up sampling a random Δ and sharing that state across simulator invocations to generate valid labels for the wires.)

3.2 Existing Half-Gates Proof and how it is to be modelled into state separating proofs

Chapter 4

Domino

In this chapter, we introduce Domino, a proof-helper for State-Separating Proofs. Then, we discuss how to set a environment up with Domino, and finally, attempt to verify the proof from Section 3 in Domino.

4.1 What is Domino?

What is domino and how does it work? The layers that exist, and the design in general.

Domino is a proof assistant for State-separating proofs (SSPs) that allows the user to validate a SSP using familiar synatax, and an underlying SMT solver. It uses very similar syntax and constructs as SSPs, such as packages, games and theorems. The explicit relation between these games is defined using invariant files, written in the SMT-LIB language. These games are then checked against different lemmata that are needed to fulfil the relation that the user wants to establish.

Domino is published as a rust package available on Github. The domino program/crate itself contains a compiler that has its own defined language that intentionally tries to be very similar the SSP syntax defined in [BDF+18].

4.2 Using Domino

4.2.1 Dependencies

Since Domino is published as a Rust package, it requires a Rust toolchain, which can be installed using *rustup*. Rustup manages multiple Rust toolchains, and is the most straightforward way to getting set up on Linux. Rustup is typically installed using a bash script from the official website, but it is also available on most system package managers. It can also be built from source. Installing rustup automatically installs the Rust binary and cargo (the package manager), among other components.

These can also be obtained without rustup, and the installation page explains the methods in more detail [Rus].

The second prerequisite to run Domino is a functioning CVC5 installation, and for the `cvc5` executable to be added to `PATH`, so that Domino can invoke it while running. CVC5 can be built from source following the instructions in [CVC]. CVC5 is the SMT solver that checks whether the constraints given by our proof are satisfiable or not. This is done in practice by taking the negation of the condition we want, and checking for satisfiability of that negation. If it exists, the condition is false.

4.2.2 Project Structure

A Domino project directory requires the presence of an empty *spp.toml* file. It allows SSP logic to be written into packages, games and theorem files. These are all placed under their own directories, i.e., folders called *packages*, *games*, and *theorem*.

Each of these folders can then contain one or more files. Domino checks packages and games for syntactic correctness even if they are not used in a theorem. This is intentional design, since when writing state separating proofs, the components (packages and games) are usually set up before the actual proof.

In older versions of Domino, theorems were called proofs, and the folder and file header for proofs were named as such.

An Example Theorem in Domino

mIND-CPA game here, talk about its packages, and the two games with diagrams, and how the theorem to prove states that the two games are indistinguishable.

4.3 Coding Half-gates in Domino

4.4 Improvements

Chapter 5

Discussion

In this chapter, we discuss the advantages and challenges when using Domino compared to a pen-and-paper proof.

Chapter 6

Conclusion

References

- [AL10] Y. Aumann and Y. Lindell, “Security against covert adversaries: Efficient protocols for realistic adversaries”, *Journal of Cryptology*, vol. 23, no. 2, pp. 281–343, Apr. 2010.
- [App16] B. Applebaum, “Garbling XOR gates “for free” in the standard model”, *Journal of Cryptology*, vol. 29, no. 3, pp. 552–576, Jul. 2016.
- [BDE+22] C. Brzuska, A. Delignat-Lavaud, et al., “Key-schedule security for the TLS 1.3 standard”, in *Advances in Cryptology – ASIACRYPT 2022, Part I*, (Taipei, Taiwan, Dec. 5–9, 2022), S. Agrawal and D. Lin, Eds., ser. Lecture Notes in Computer Science, vol. 13791, Springer, Cham, Switzerland, 2022, pp. 621–650.
- [BDF+18] C. Brzuska, A. Delignat-Lavaud, et al., *State separation for code-based game-playing proofs*, Publication info: A major revision of an IACR publication in ASIACRYPT 2018, 2018. last visited: Apr. 10, 2026. [Online]. Available: <https://eprint.iacr.org/2018/306>
- [BHR12] M. Bellare, V. T. Hoang, and P. Rogaway, *Foundations of garbled circuits*, Publication info: Published elsewhere. Proceeding version appears in CCS 2012, 2012. last visited: Jan. 14, 2026. [Online]. Available: <https://eprint.iacr.org/2012/265>
- [BMR90] D. Beaver, S. Micali, and P. Rogaway, “The round complexity of secure protocols (extended abstract)”, in *22nd Annual ACM Symposium on Theory of Computing*, (Baltimore, MD, USA, May 14–16, 1990), ACM Press, 1990, pp. 503–513.
- [BO23] C. Brzuska and S. Oechsner, “A state-separating proof for Yao’s garbling scheme”, in *CSF 2023: IEEE 36th Computer Security Foundations Symposium*, (Dubrovnik, Croatia, Jul. 10–14, 2023), IEEE Computer Society Press, 2023, pp. 137–152.
- [BW] R. Brzuska Egger and Winkermann, private communication.
- [CVC] CVC5. “Installation — cvc5 documentation”, last visited: May 25, 2026. [Online]. Available: <https://cvc5.github.io/docs/cvc5-1.0.0/installation/installation.html>

- [EGL82] S. Even, O. Goldreich, and A. Lempel, “A randomized protocol for signing contracts”, in *Advances in Cryptology – CRYPTO’82*, D. Chaum, R. L. Rivest, and A. T. Sherman, Eds., Santa Barbara, CA, USA: Plenum Press, New York, USA, 1982, pp. 205–210.
- [HYLL] N. Haloani, A. Yanai, et al., “COTI V2: Confidential Computing Ethereum Layer 2”, COTI, Soda Labs, Tech. Rep. [Online]. Available: https://coti.io/files/coti_v2_whitepaper.pdf
- [KS08] V. Kolesnikov and T. Schneider, “Improved garbled circuit: Free XOR gates and applications”, in *Automata, Languages and Programming*, L. Aceto, I. Damgård, et al., Eds., Berlin, Heidelberg: Springer Berlin Heidelberg, 2008, pp. 486–498.
- [Lin17] Y. Lindell, “How to simulate it – a tutorial on the simulation proof technique”, in *Tutorials on the Foundations of Cryptography*, Y. Lindell, Ed., Series Title: Information Security and Cryptography, Cham: Springer International Publishing, 2017, pp. 277–346. last visited: May 21, 2026. [Online]. Available: http://link.springer.com/10.1007/978-3-319-57048-8_6
- [LP09] Y. Lindell and B. Pinkas, “A proof of security of Yao’s protocol for two-party computation”, *Journal of Cryptology*, vol. 22, no. 2, pp. 161–188, Apr. 2009.
- [MNPS04] D. Malkhi, N. Nisan, et al., “Fairplay—a secure two-party computation system”, in *Proceedings of the 13th conference on USENIX Security Symposium - Volume 13*, ser. SSYM’04, USA: USENIX Association, Aug. 13, 2004, p. 20. last visited: Jun. 25, 2026.
- [PtC16] E. Parliament and the Council of the European Union, *Regulation (EU) 2016/679 of the European Parliament and of the Council of 27 April 2016 on the protection of natural persons with regard to the processing of personal data and on the free movement of such data, and repealing Directive 95/46/EC (General Data Protection Regulation) (Text with EEA relevance)*, May 2016. [Online]. Available: <http://data.europa.eu/eli/reg/2016/679/oj>
- [Raj25] A. Rajabi, “Towards formally verifying the TLS 1.3 Key Schedule Security in SSBe”, en, *Master thesis, Aalto University, 2025*, Jul. 2025. [Online]. Available: <https://aaltodoc.aalto.fi/handle/123456789/138153>
- [Rus] Rust Foundation. “Other Installation Methods - Rust Forge”, last visited: May 25, 2026. [Online]. Available: <https://forge.rust-lang.org/infra/other-installation-methods.html>
- [Yao86] A. C.-C. Yao, “How to generate and exchange secrets (extended abstract)”, in *27th Annual Symposium on Foundations of Computer Science*, (Toronto, Ontario, Canada, Oct. 27–29, 1986), IEEE Computer Society Press, 1986, pp. 162–167.
- [ZRE15] S. Zahur, M. Rosulek, and D. Evans, “Two halves make a whole - reducing data transfer in garbled circuits using half gates”, in *Advances in Cryptology – EUROCRYPT 2015, Part II*, (Sofia, Bulgaria, Apr. 26–30, 2015), E. Oswald and M. Fischlin, Eds., ser. Lecture Notes in Computer Science, vol. 9057, Springer Berlin Heidelberg, Germany, 2015, pp. 220–250.